SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

삼성청년
SW·AI아카데미

backend - servlet

SAMSUNG SW AI ACADEMY FOR YOUTH

목차

1. Backend Programming
2. Maven
3. Servlet

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Backend Programming

개발 환경 점검

Confidential

❖ STS 설정

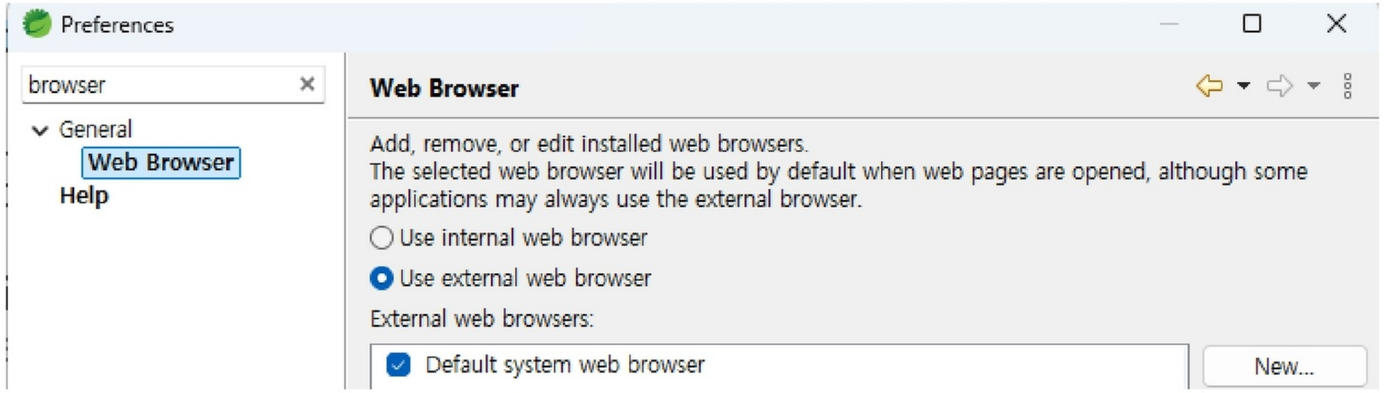
- 플러그인 설치
  - ◆ help > Eclipse Marketplace > Eclipse Enterprise Java and Web Developer Tools 설치

개발 환경 점검

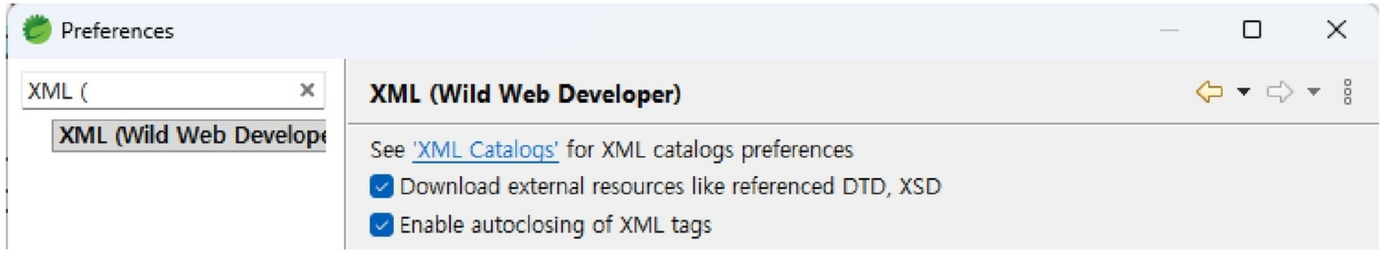
Confidential

❖ STS 설정

- 외부 브라우저 연결
  - ◆ preferences > browser 검색 > Web Browser > Use external web browser 선택



- 외부 DTD 다운로드 설정
  - ◆ XML(Wild Web Developer) > download external like referenced DTD, XSD 설정



개발 환경 점검

Confidential

❖ WAS 설치

- https://tomcat.apache.org > Download > Tomcat 11

11.0.18

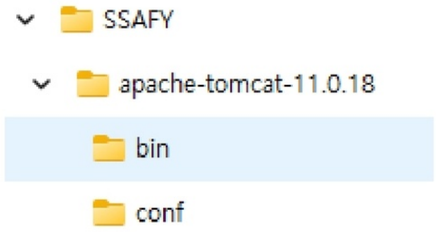
Please see the README file for packaging information. It explains what every distribution contains.

Binary Distributions

- Core:
  - zip (pgp, sha512)
  - tar.gz (pgp, sha512)
  - Windows zip (pgp, sha512)
  - Windows Service Installer (pgp, sha512)

11.X final release!

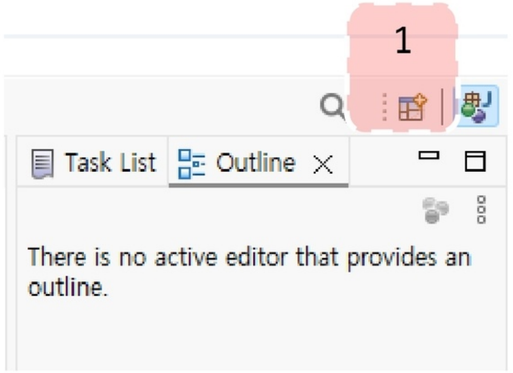
- C:₩SSAFY에 파일 배치 후 “여기서 압축 풀기”

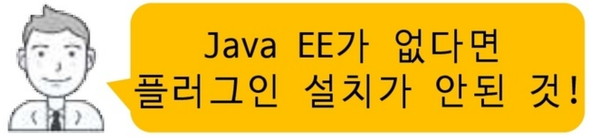


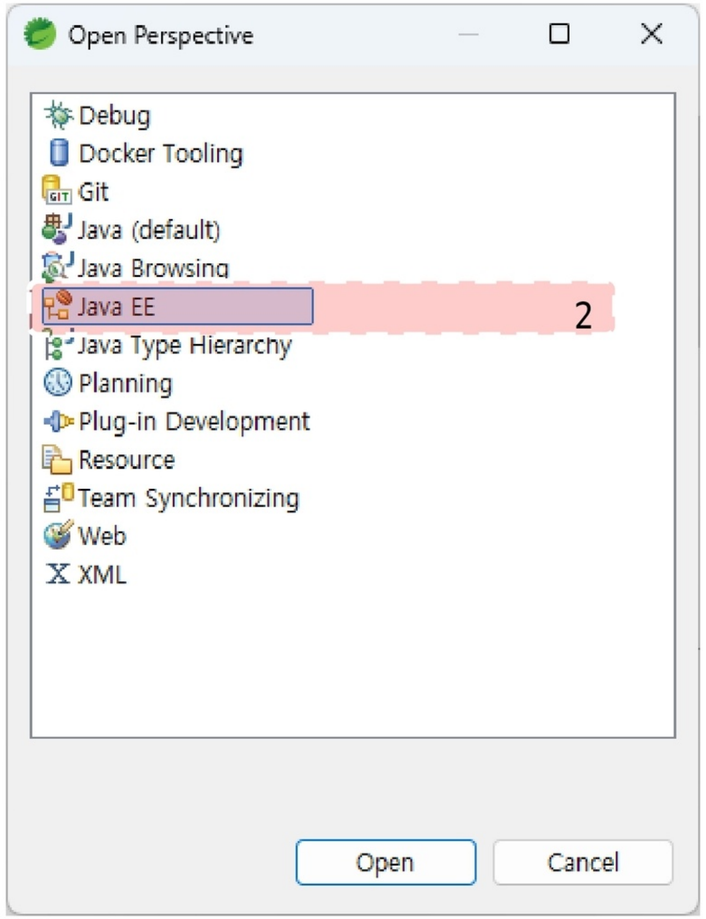
Confidential

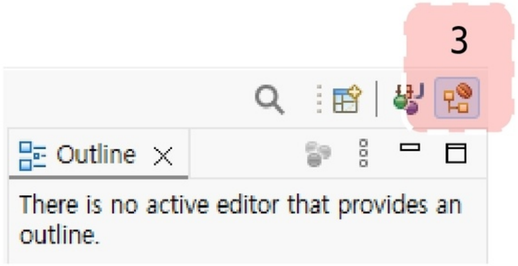
❖ Jakarta EE(J2EE) 환경 설정

- open perspective > Java EE 선택
  - ◆ Perspective: 관련 프로젝트를 위한 화면(view)의 모음







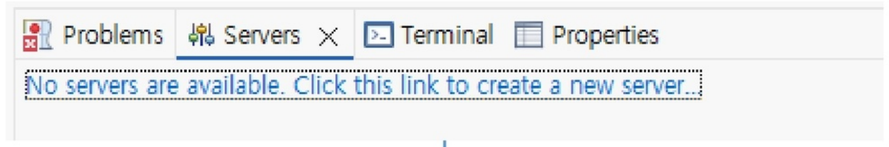


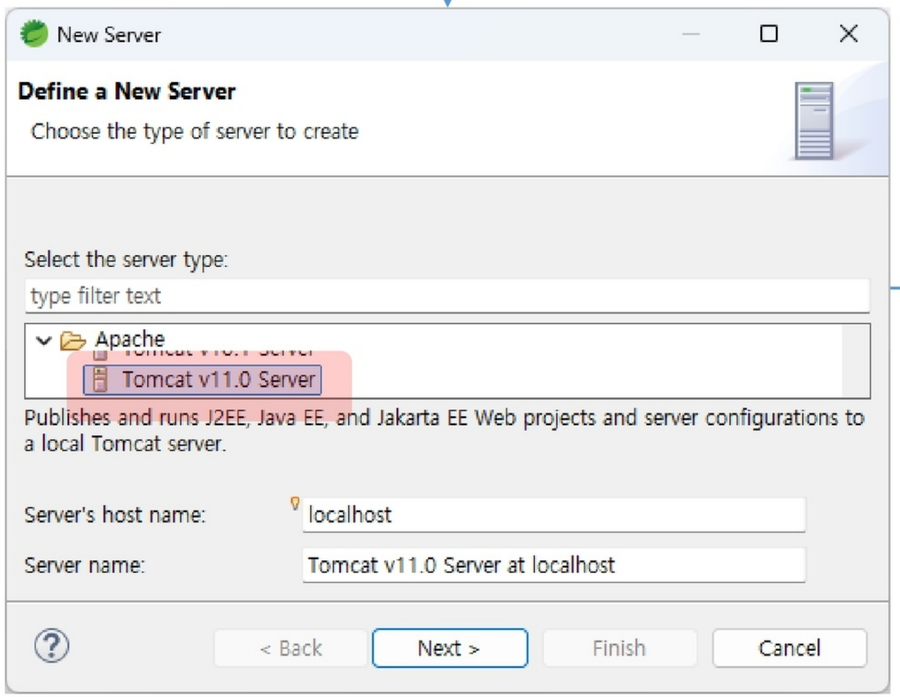
개발 환경 점검

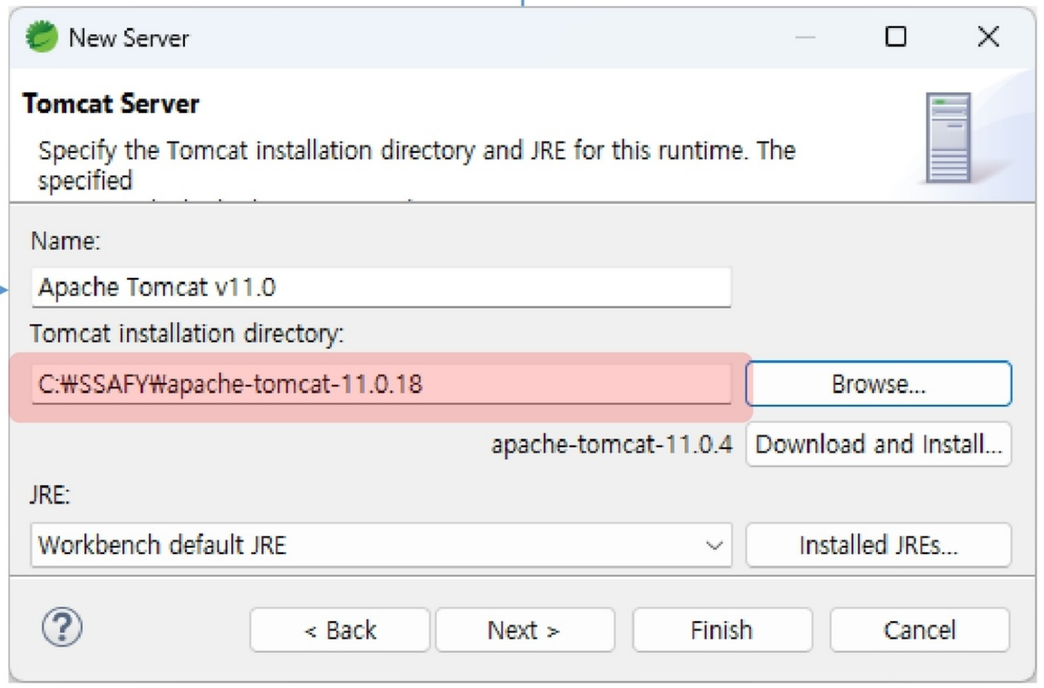
Confidential

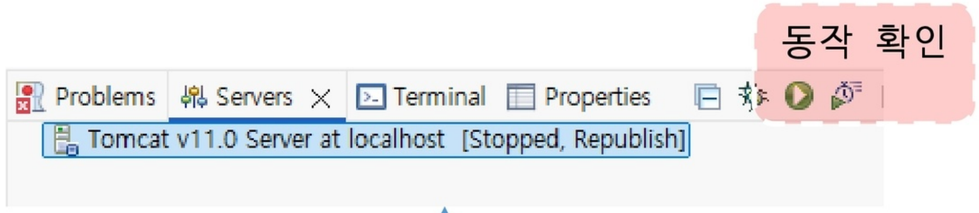
❖ 서버 런타임 확인

- Servers 탭에서 runtime server 확인





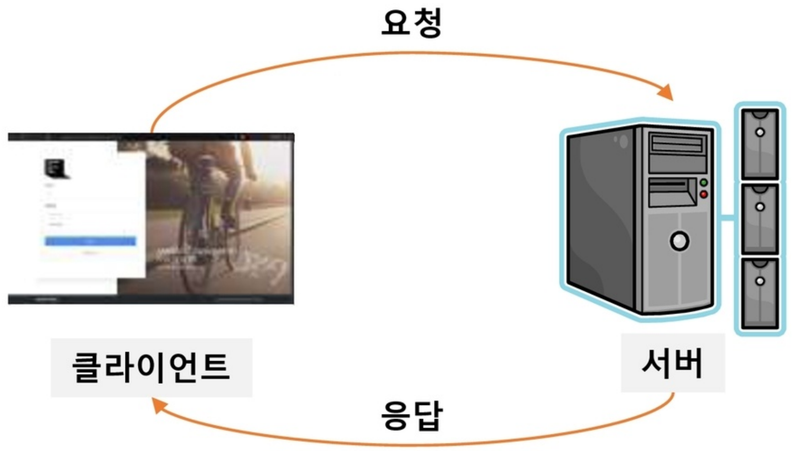


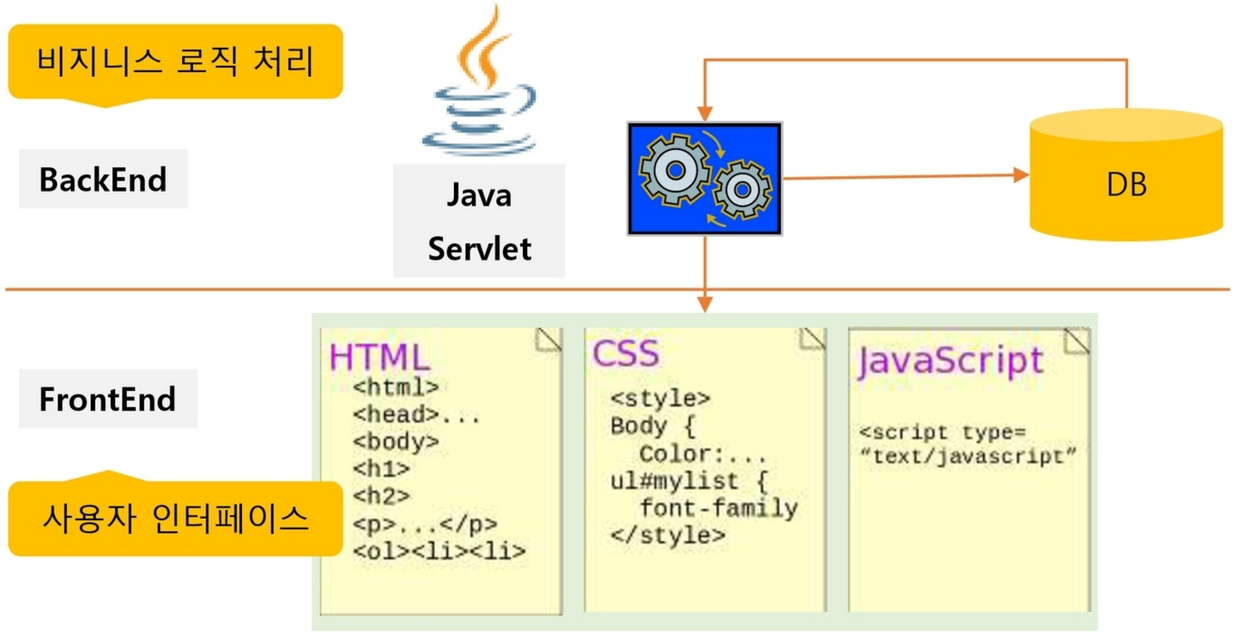


Confidential

❖ 웹 프로그램

- 네트워크 너머의 서버에서 존재하며 웹을 통해서 서비스되는 프로그램
- 비즈니스 로직을 위한 backend와 사용자 인터페이스를 위한 frontend로 구성





Content
- 페이지 구조와 내용
- 이미지 링크...
- 기본적 상호작용

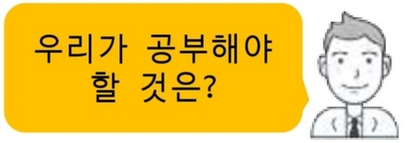
Presentation
- 페이지 디자인
- 컬러, 폰트...
- 위치, 배치...

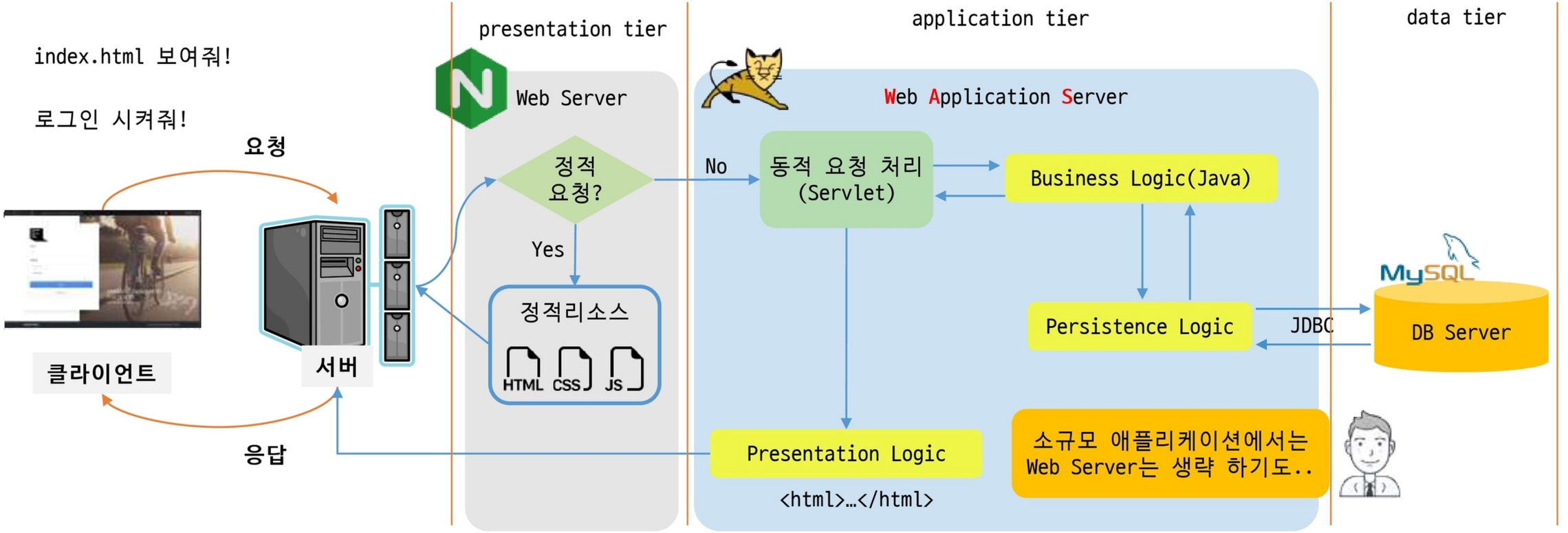
Behavior
- 이벤트 처리
- 고수준 상호작용
- 에러 체크

Confidential

❖ 3-Tier Architecture

- 애플리케이션을 3개의 논리적 및 물리적 컴퓨팅 계층으로 분리해서 운영
  - ◆ presentation tier, application tier, data tier
  - ◆ 각 계층은 자체 인프라에서 실행되기 때문에 별도로 개발, 운영
  - ◆ 다른 계층에 영향을 주지 않고 필요에 따라 업데이트 및 확장 가능



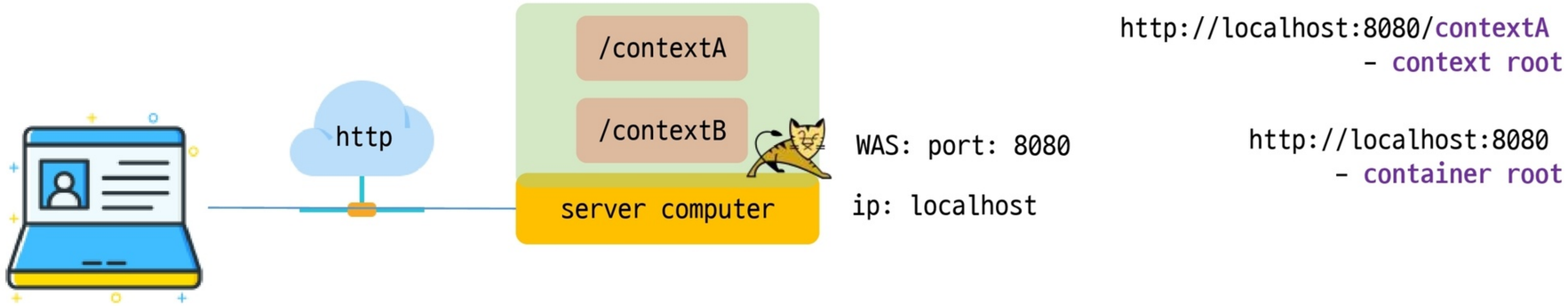


Container와 Context

Confidential

❖ WAS, Container, Context

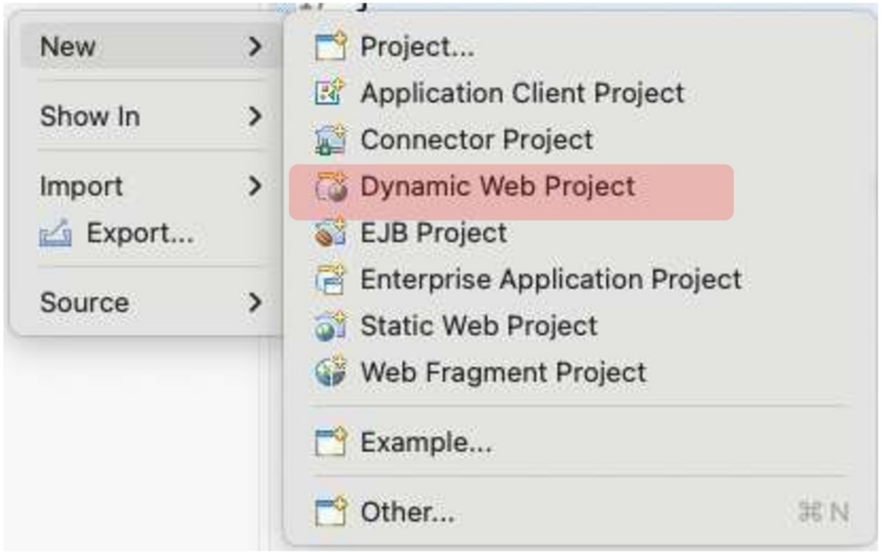
- 물리적인 서버는 network에서 접근하기 위해서 ip 주소를 갖음 - HTTP 기반으로 통신
  - ◆ 물리적인 서버에는 WAS가 설치되어 HTTP 기반의 웹 서비스 처리
- WAS는 일반적으로 80 port에서 웹 서비스를 제공하며 개발용으로는 8080을 주로 사용
  - ◆ WAS는 내부에 Servlet/JSP를 실행하는 엔진 (servlet container)를 포함 ➔ WAS ≈ Container
- WAS에는 동시에 여러 가지 웹 애플리케이션이 동작 가능
  - ◆ 각 **애플리케이션의 실행 환경과 실행 정보 제공하는 것**을 **Context**라고 함
  - ◆ 엄밀히는 기능을 제공하는 application과 실행 환경으로 다르지만 거의 동일한 개념으로 사용



Confidential

❖ 프로젝트 생성

- Dynamic Web Project 생성

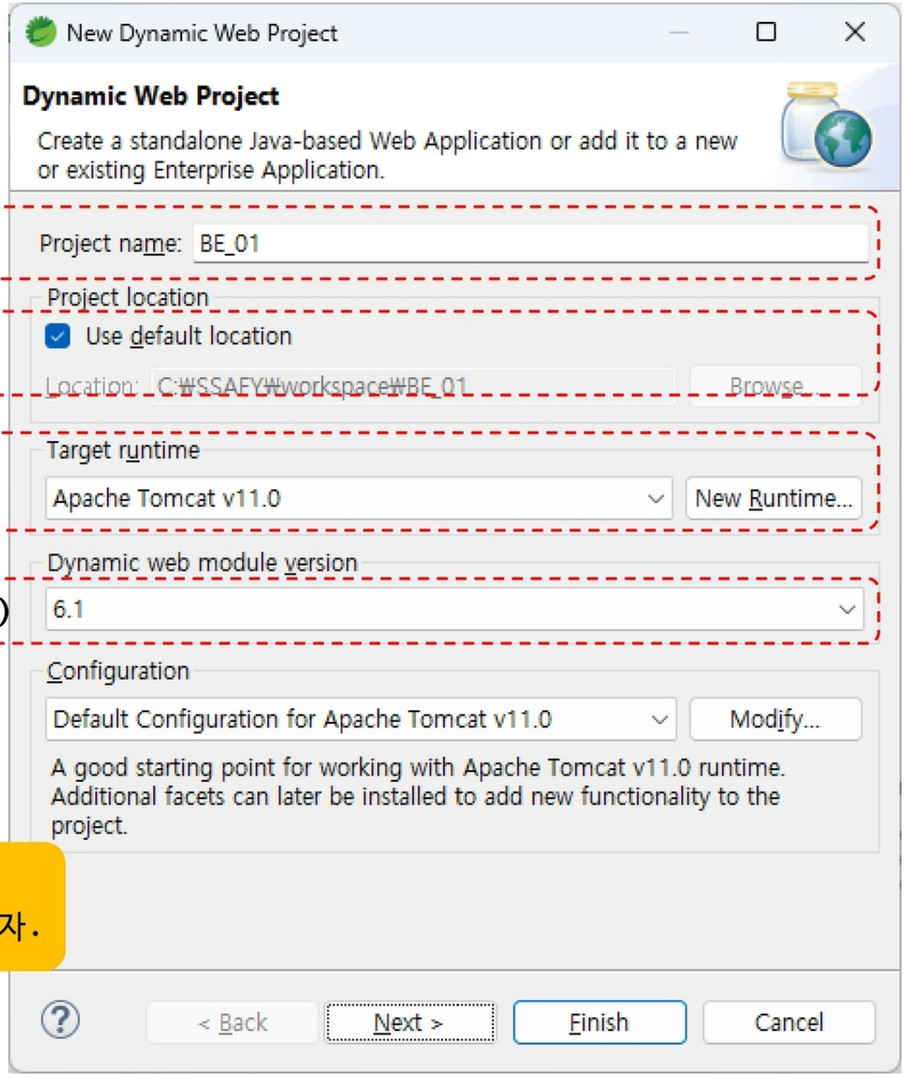


프로젝트 이름 설정

프로젝트 경로 확인

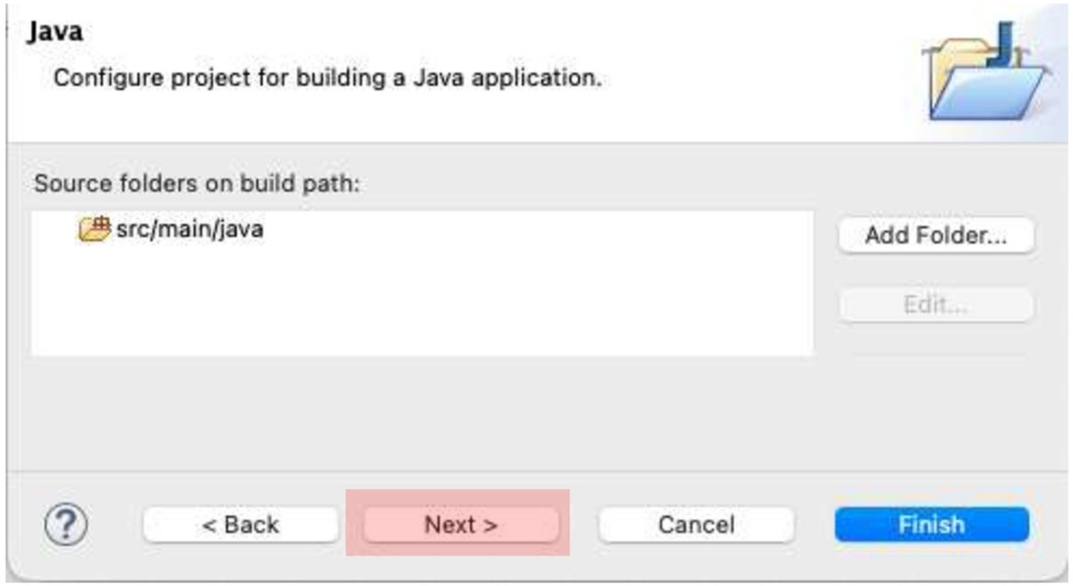
런타임 서버 확인

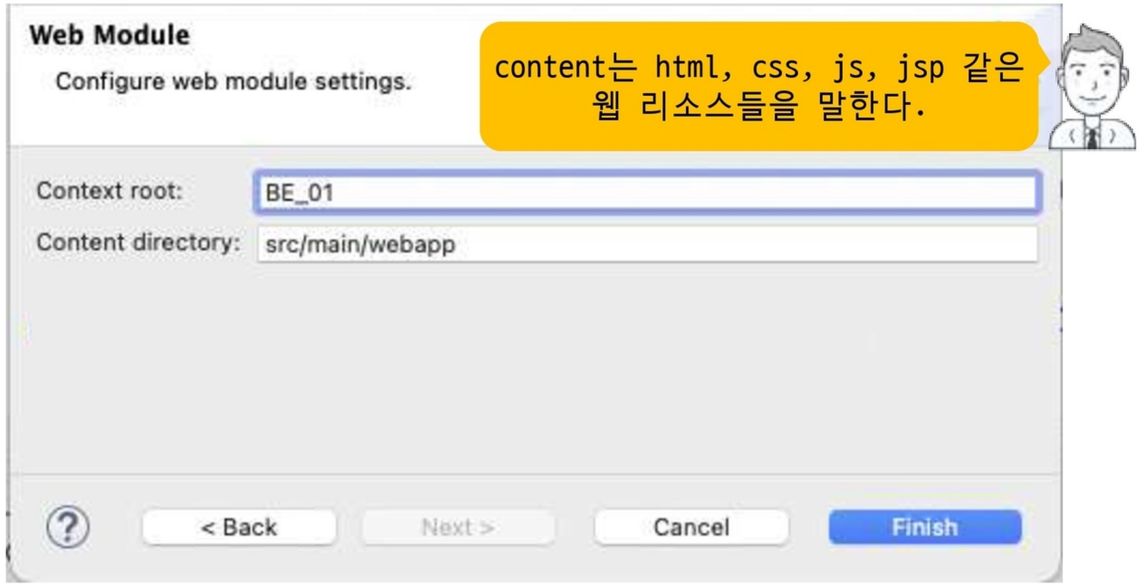
웹 모듈 버전 확인(6.1)



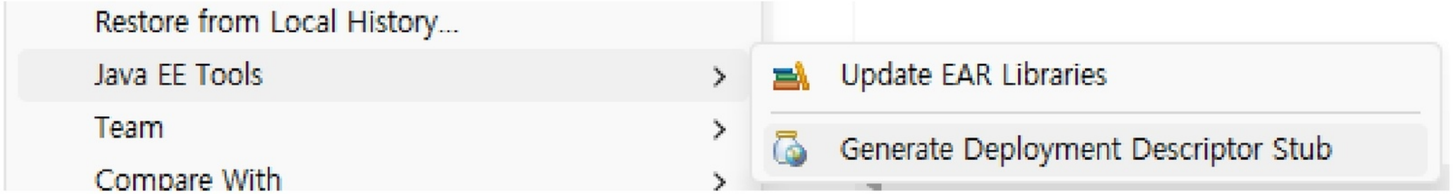
Finish 말고
Next를 클릭해보자.

❖ 프로젝트 생성



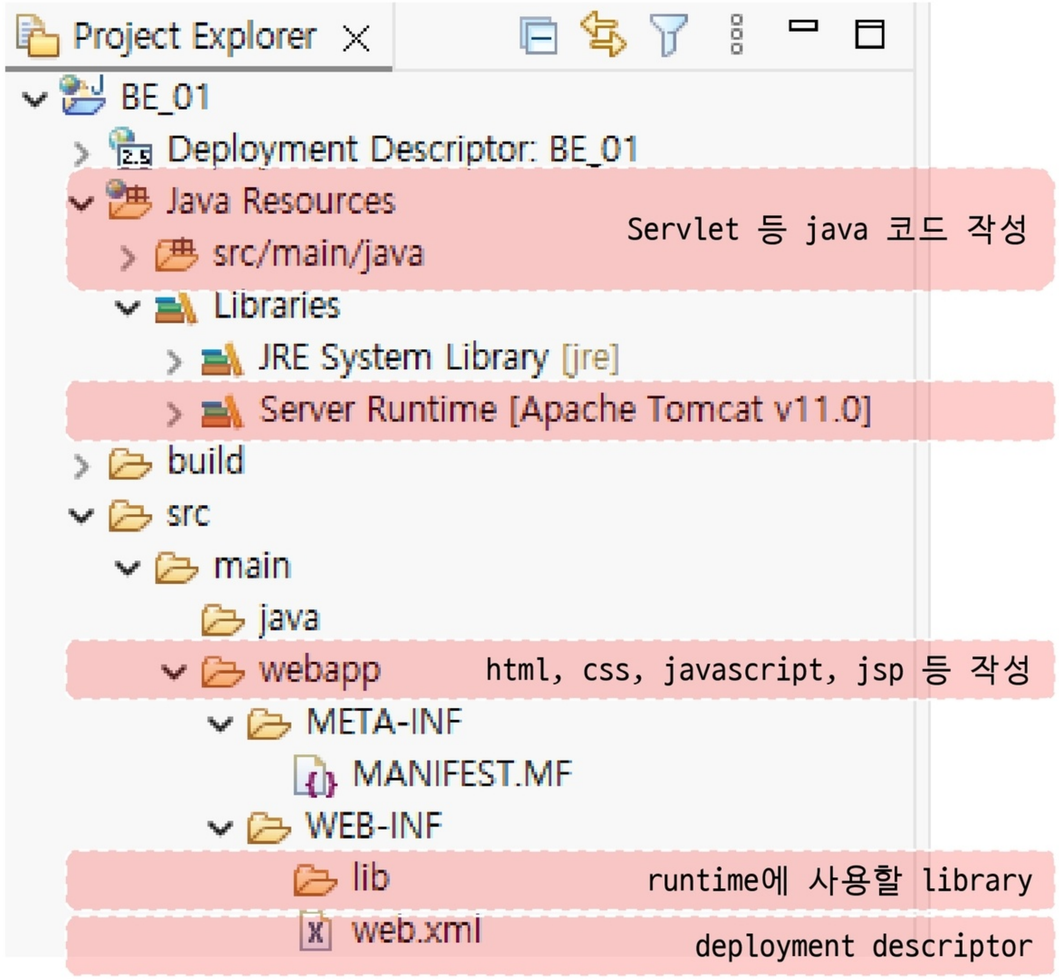


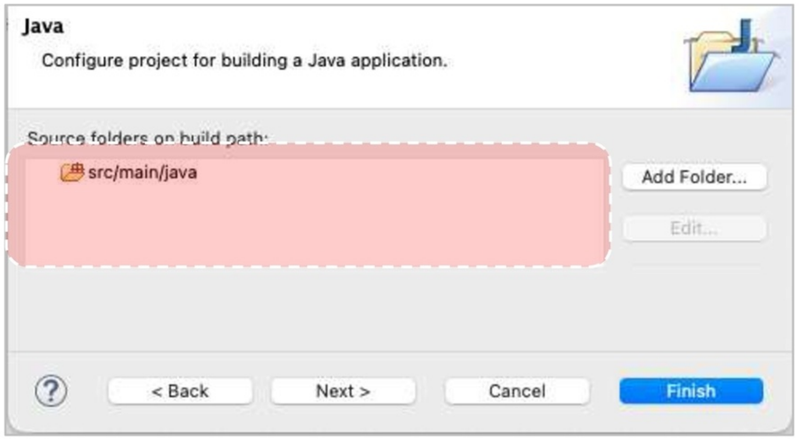
- src/main/webapp/WEB-INF/web.xml
  - ◆ 없으면 생성: project context menu > Java EE Tools > Generate Deployment Descriptor Stub

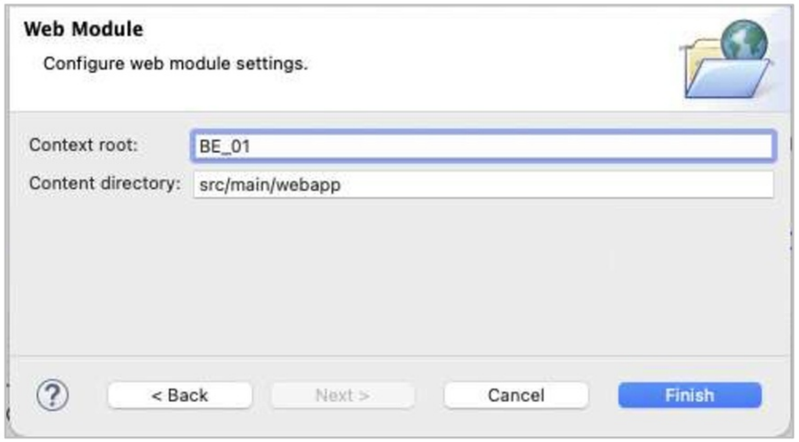


Confidential

❖ 프로젝트 기본 구조





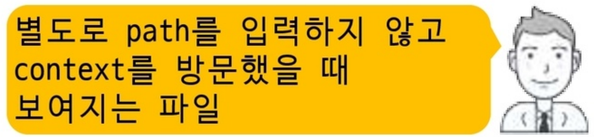


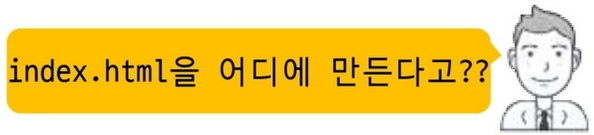
Confidential

❖ Deployment descriptor

- web.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="https://jakarta.ee/xml/ns/jakartaee"
xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
id="WebApp_ID" version="6.0">
<display-name>HelloServlet</display-name>
<welcome-file-list>
  <welcome-file>index.html</welcome-file>
  <welcome-file>index.jsp</welcome-file>
  <welcome-file>index.htm</welcome-file>
  <welcome-file>default.html</welcome-file>
  <welcome-file>default.jsp</welcome-file>
  <welcome-file>default.htm</welcome-file>
</welcome-file-list>
</web-app>
```

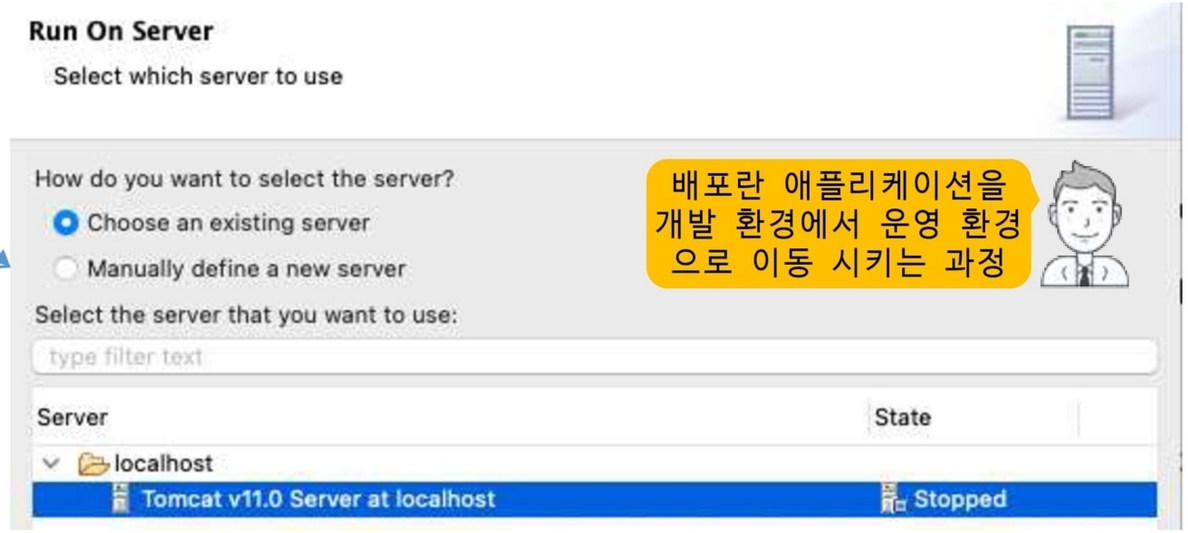




Confidential

❖ 실행

배포

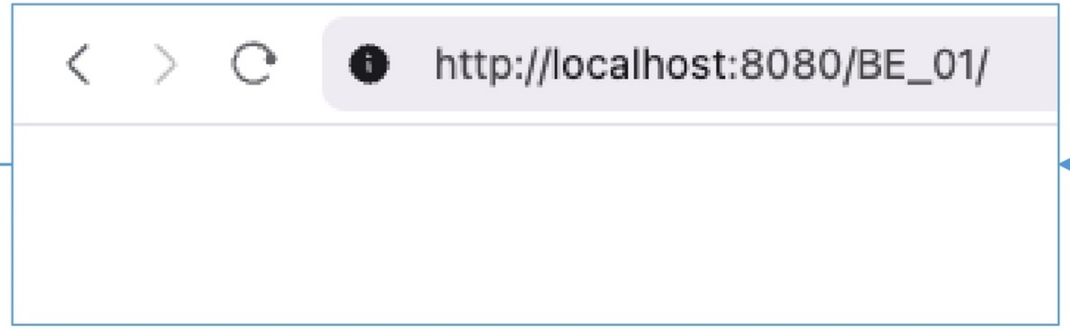


응답:

```
<html>Servlet Day 1</html>
```

요청:

```
http:    //localhost:8080/BE_01          /index.html
protocol://container_root/context_root  /url_mapping
```

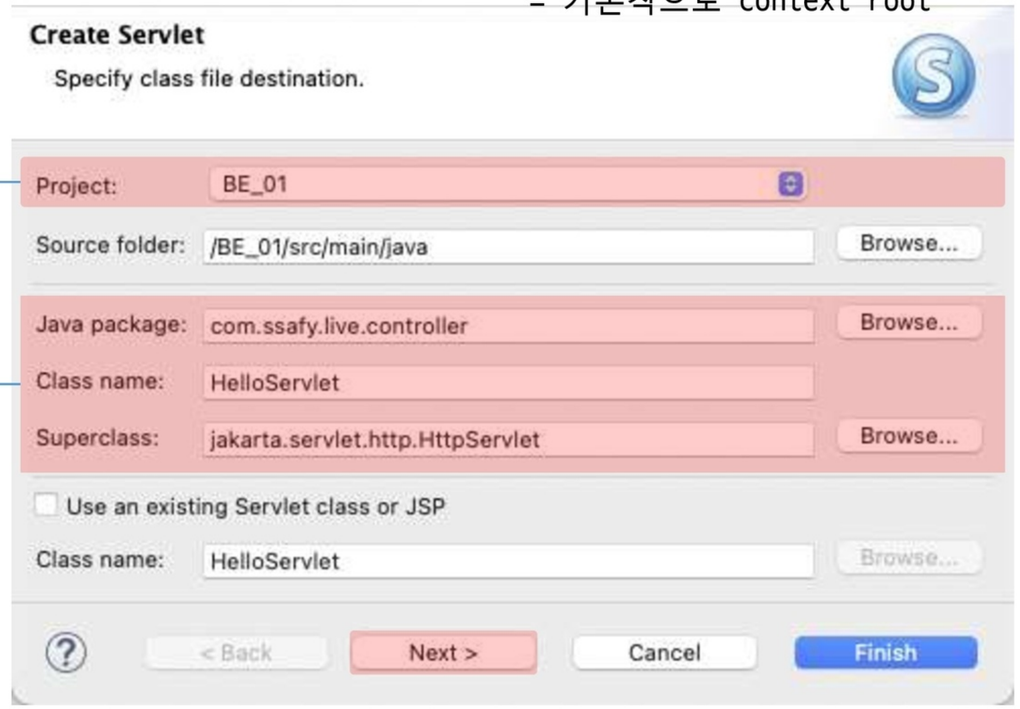


Confidential

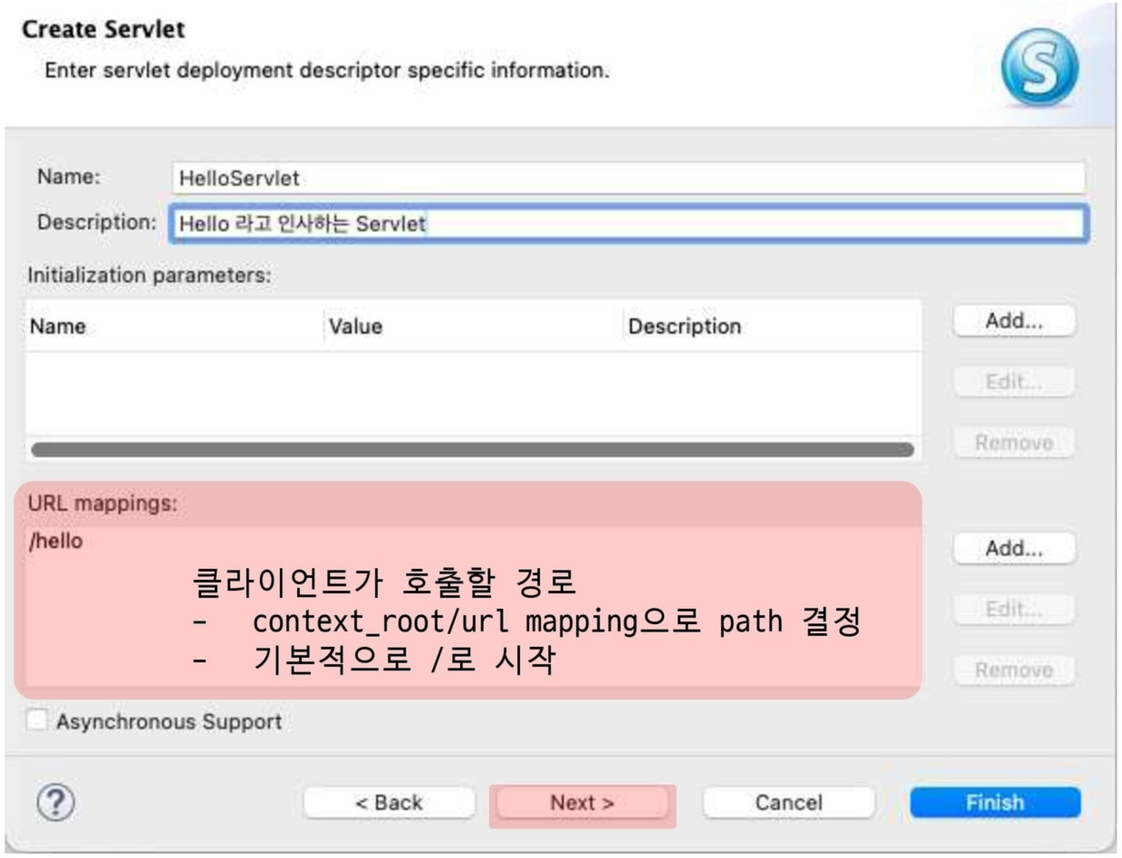
❖ 새로운 Servlet 생성

- new -> Servlet

project name
= 기본적으로 context root



- HttpServlet 상속!
- 사용자가 패키지, 클래스 이름을 알 수 있을리가..

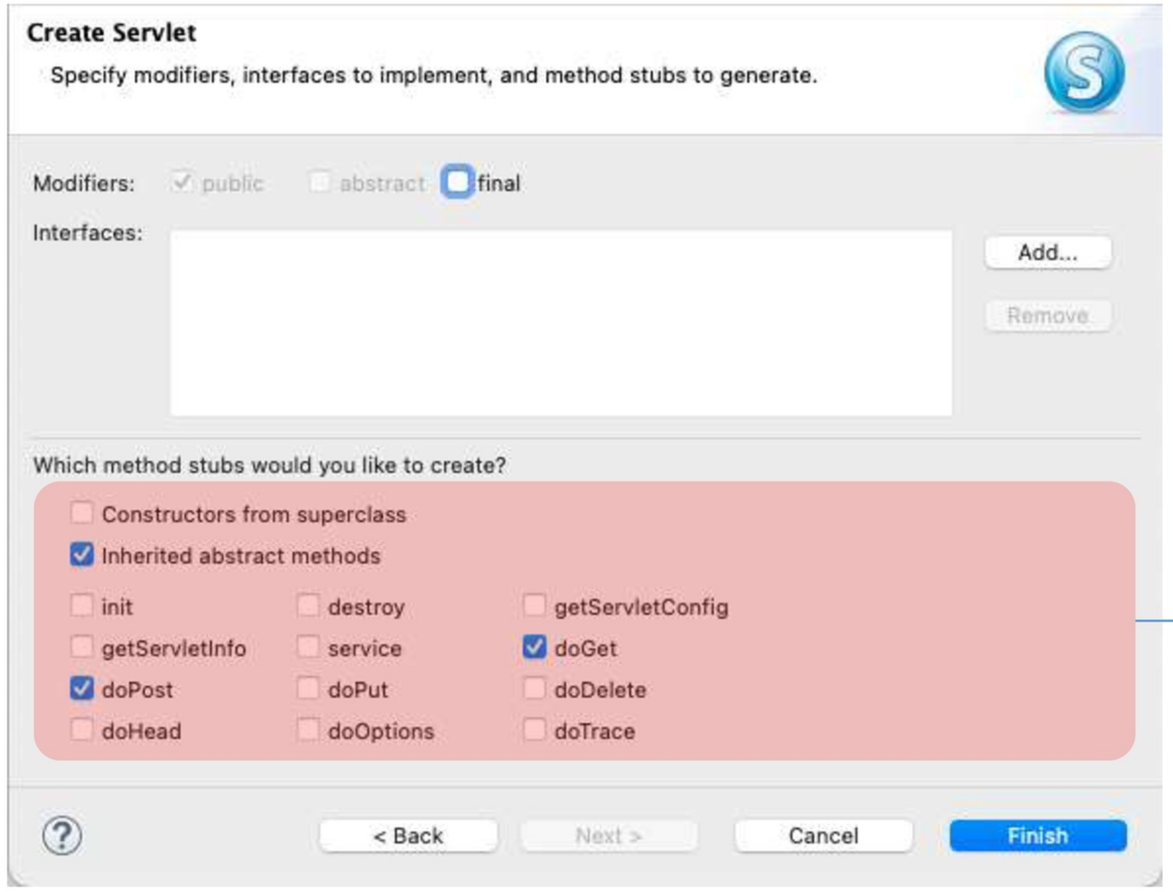


클라이언트가 호출할 경로
- context_root/url mapping으로 path 결정
- 기본적으로 /로 시작

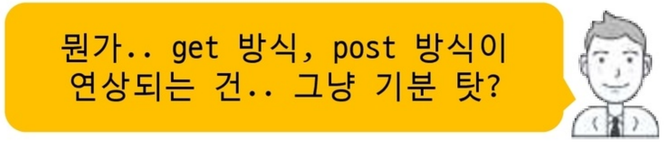
Confidential

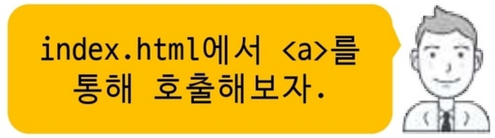
❖ 새로운 Servlet 생성

- new -> Servlet



필요한 메서드 재정의
- doGet과 doPost





SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Maven

Confidential

❖ maven을 이용한 프로젝트 관리

- project의 규모가 커지면서 사용해야 하는 라이브러리들이 다양해짐 - 팀원들과 버전 동기화 문제 발생

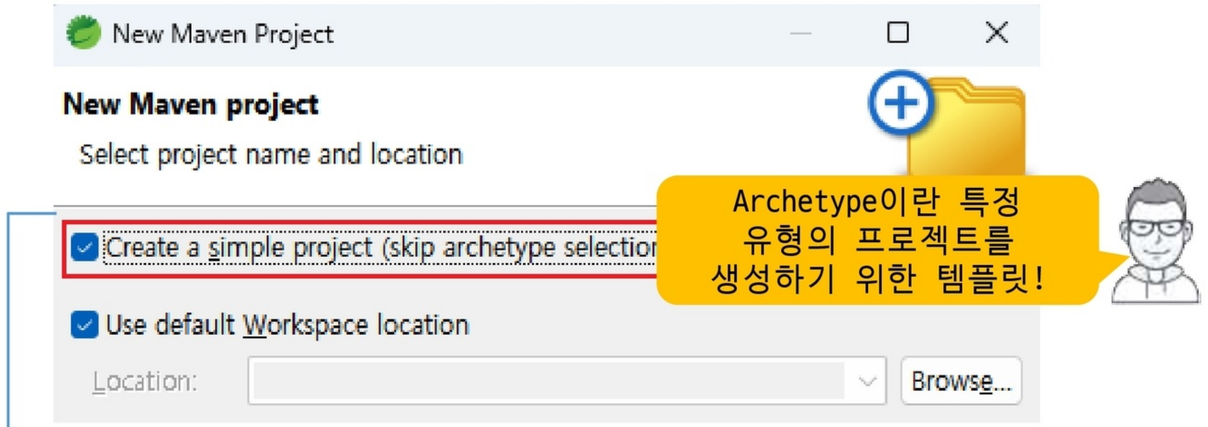
- maven: Java 기반의 프로젝트 관리 도구로 주로 빌드 자동화와 의존성 관리에 사용

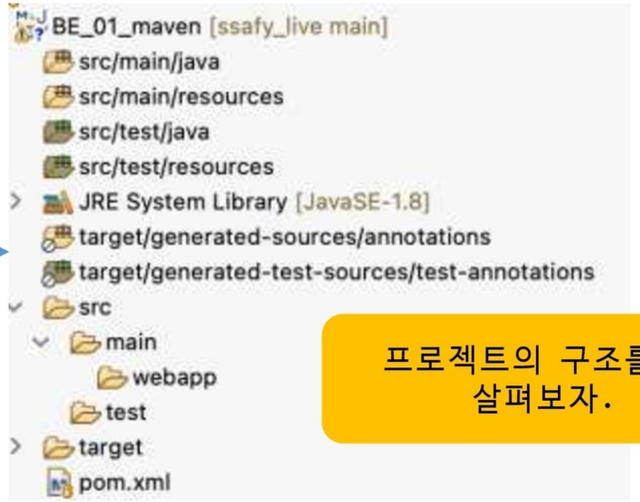
- maven의 주요 기능
  - ◆ 프로젝트 구조 표준화
  - ◆ 의존성 관리
  - ◆ 빌드 자동화

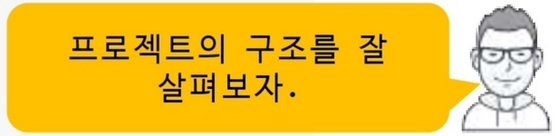
Confidential

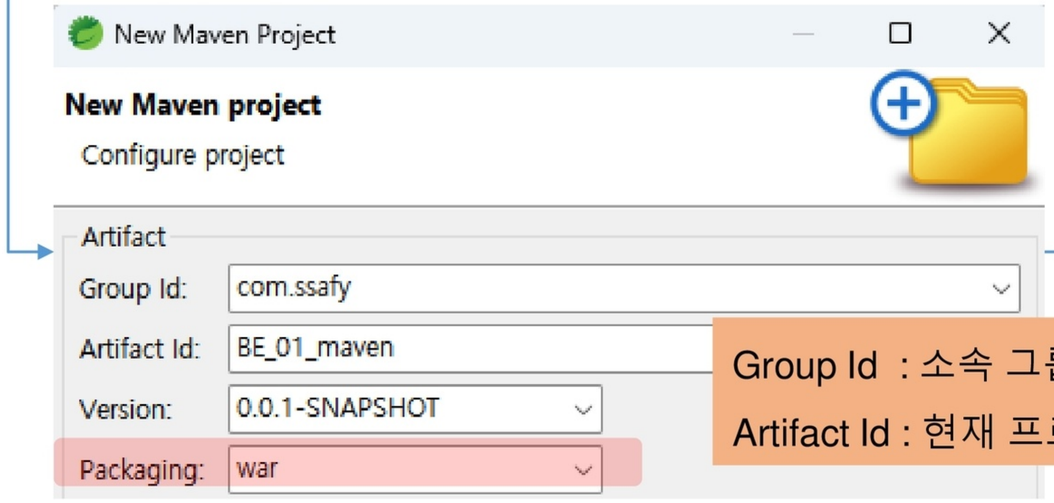
❖ project 생성

- new > Maven Project









Group Id : 소속 그룹의 identity

Artifact Id : 현재 프로젝트 identity

Confidential

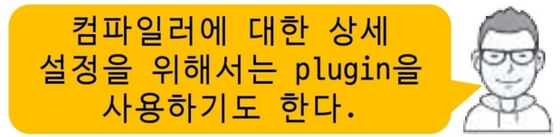
❖ project 수정

● JDK 버전 변경

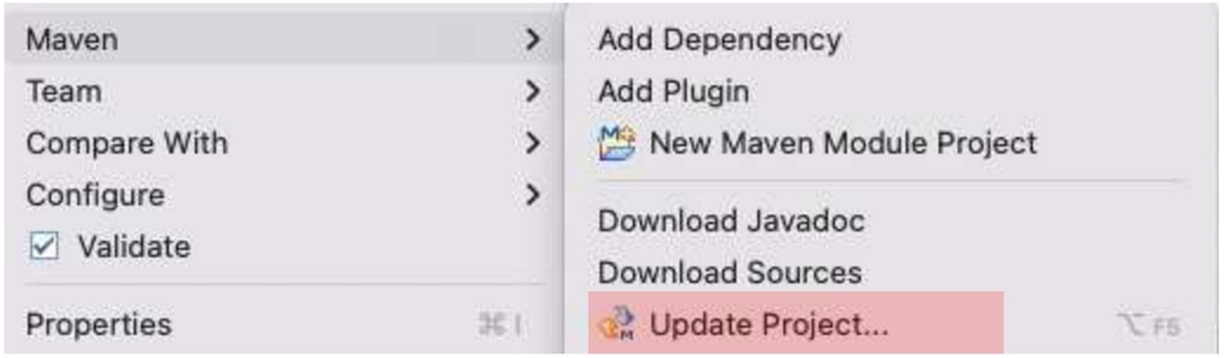
```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.ssafy</groupId>
  <artifactId>BE_01_maven</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <properties>
    <jdk.version>프로젝트_JDK_VERSION</jdk.version>
    <maven.compiler.release>${jdk.version}</maven.compiler.release>
  </properties>
</project>
```

커스텀 태그로 변수 선언

${}로 변수 참조



◆ Project Context menu > maven > update project



Confidential

❖ 라이브러리 추가

- https://mvnrepository.com/

1. **Jakarta Servlet API** — 10,035 usages

jakarta.servlet » jakarta.servlet-api — EPL +1

API for Jakarta Servlet, enabling web applications to handle HTTP requests and responses through a standard interface for building dynamic web content.

Last Release on Jun 11, 2024

3. **Jakarta Pages API** — 528 usages

jakarta.servlet.jsp » jakarta.servlet.jsp-api — EPL +1

Jakarta Pages defines a templating technology for dynamically creating web resources

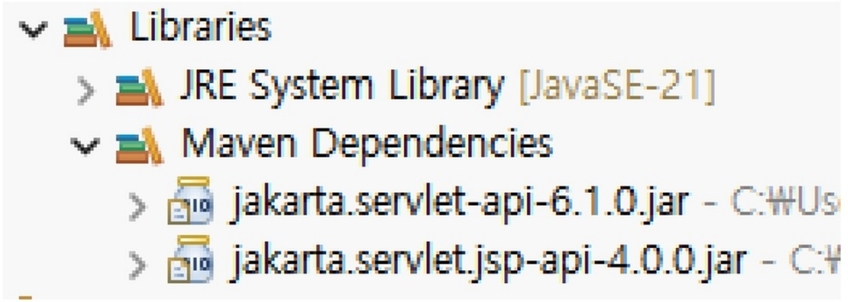
Last Release on Jun 11, 2024

Confidential

❖ maven 기반 프로젝트 생성

- pom.xml 파일 수정

```
<dependencies>
  <dependency>
    <groupId>jakarta.servlet</groupId>
    <artifactId>jakarta.servlet-api</artifactId>
    <version>6.1.0</version>
  </dependency>

  <dependency>
    <groupId>jakarta.servlet.jsp</groupId>
    <artifactId>jakarta.servlet.jsp-api</artifactId>
    <version>4.0.0</version>
  </dependency>
</dependencies>
```



Confidential

❖ maven 기반 프로젝트 생성

- 〈properties〉
  - ◆ 특히 버전 정보를 property로 선언해서 재사용성 강화

```
<properties>
  <jdk.version>프로젝트_JDK_VERSION</jdk.version>
  <maven.compiler.release>${jdk.version}</maven.compiler.release>

  <jakarta.servlet.version>6.0.0</jakarta.servlet.version>
  <jakarta.servlet.jsp-api.version>4.0.0</jakarta.servlet.jsp-api.version>
</properties>

<dependencies>
  <dependency>
    <groupId>jakarta.servlet</groupId>
    <artifactId>jakarta.servlet-api</artifactId>
    <version>${jakarta.servlet.version}</version>
    <scope>provided</scope>
  </dependency>
</dependencies>
```

SAMSUNG SW AI ACADEMY FOR YOUTH

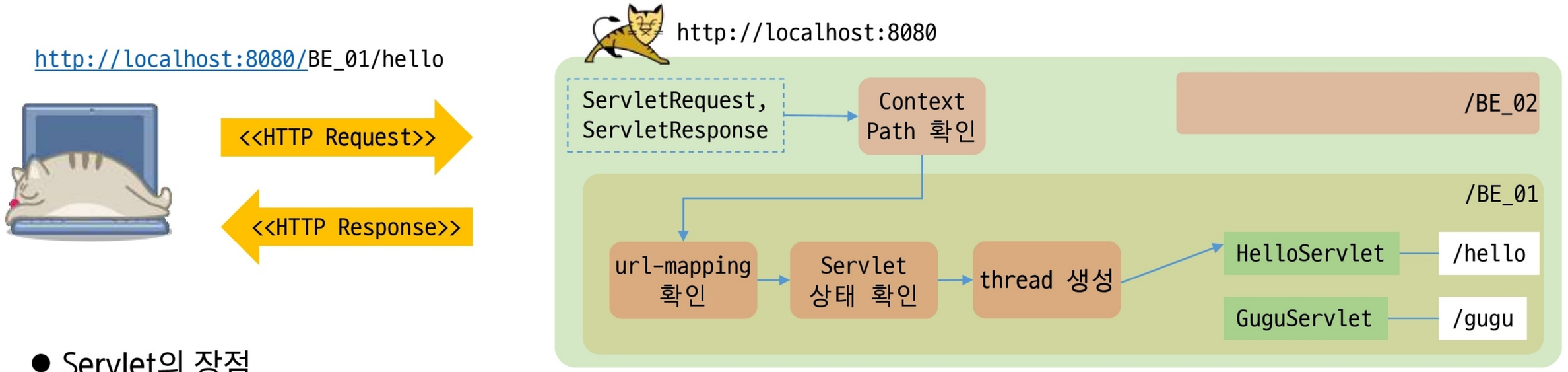
함께가요 미래로!
Enabling People

Servlet

Confidential

❖ Servlet 특징과 요청 처리 과정

- **Servlet이란? WAS에서 실행되는 Java Web Component - WAS를 servlet container라고도 한다!**



- Servlet의 장점
  - ◆ java의 OOP 기반으로 작성: 유지보수성 및 재활용성 우수하며 플랫폼 독립적
  - ◆ 높은 성능: 하나의 서블릿 객체만 생성하며 멀티스레딩을 지원하여 요청의 동시 처리가 가능
  - ◆ 확장성: 필터를 통한 공통 모듈의 전/후 처리, 리스너를 이용한 이벤트 기반 처리 및 스프링 등 다양한 프레임워크와 통합 용이
- Servlet 단점
  - ◆ business logic과 presentation logic(HTML code)가 섞여서 나타남 : Servlet + JSP의 Model 2 방식으로 처리

Confidential

❖ Hello Servlet

```
@WebServlet(description = "Hello 라고 인사하는 Servlet", urlPatterns = { "/hello" })
```

URL 매핑 설정

```
public class HelloServlet extends HttpServlet {
```

HttpServlet 상속

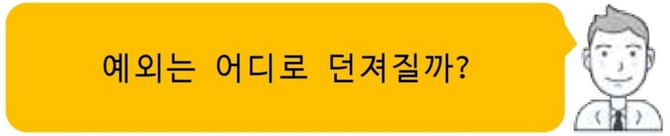
```
  protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
    response.getWriter().append("Served at: ").append(request.getContextPath());
  }
```

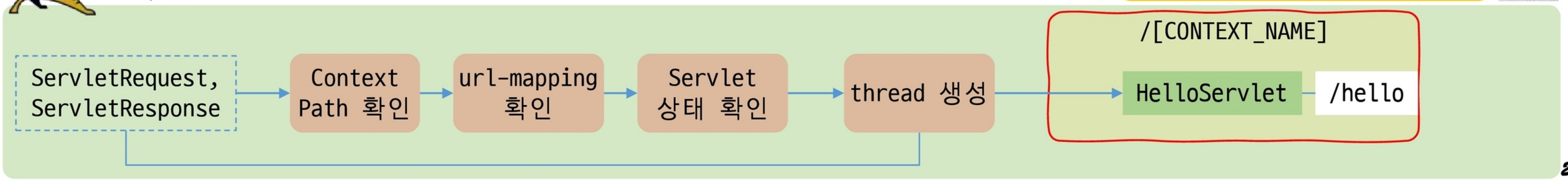
doGet 재정의

```
  protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
    doGet(request, response);
  }
}
```

doPost 재정의

http://localhost:8080

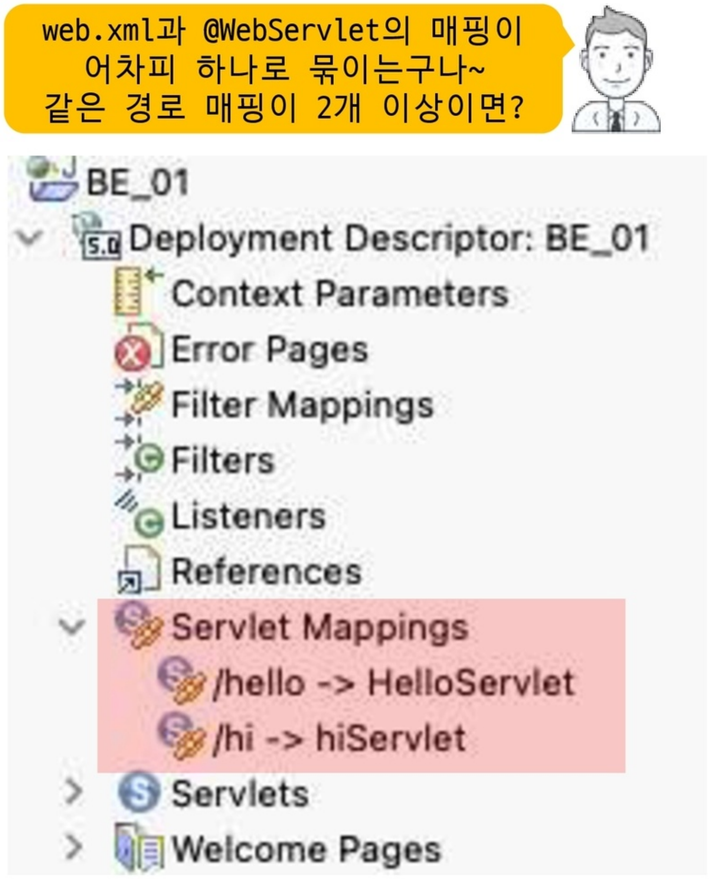




Confidential

❖ URL Mapping

- 클라이언트의 HTTP 요청을 Servlet과 매핑하기 위한 방법
- url-mapping 작성 방법
  - ◆ URL 경로 지정: / 로 시작해서 경로 지정 가능. ex) /hello/hi, /hello/there, …
  - ◆ 확장자 매칭: '*.확장자' 형태로 지정. 경로 지정과 함께 사용 불가. ex) hello.do, hi.do, …



- web.xml(deployment descriptor)를 활용한 mapping

```
<servlet>
  <servlet-name>hiServlet</servlet-name>
  <servlet-class>com.ssafy.live.controller.HelloServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>hiServlet</servlet-name>
  <url-pattern>/hi</url-pattern>
</servlet-mapping>
```

- @WebServlet 애너테이션 활용

```
@WebServlet(urlPatterns = { "/hello" })
public class HelloServlet extends HttpServlet {. . .}
```

Confidential

❖ @WebServlet 애너테이션

```
@Target({ ElementType.TYPE })
@Retention(RetentionPolicy.RUNTIME)
@Documented

public @interface WebServlet {

  String name() default "";                // The name of the servlet

  String[] value() default {};             // The URL patterns of the servlet

  String[] urlPatterns() default {};       // The URL patterns of the servlet

  int loadOnStartup() default -1;          // The load-on-startup order of the servlet

  WebInitParam[] initParams() default {}; // The init parameters of the servlet

}
```

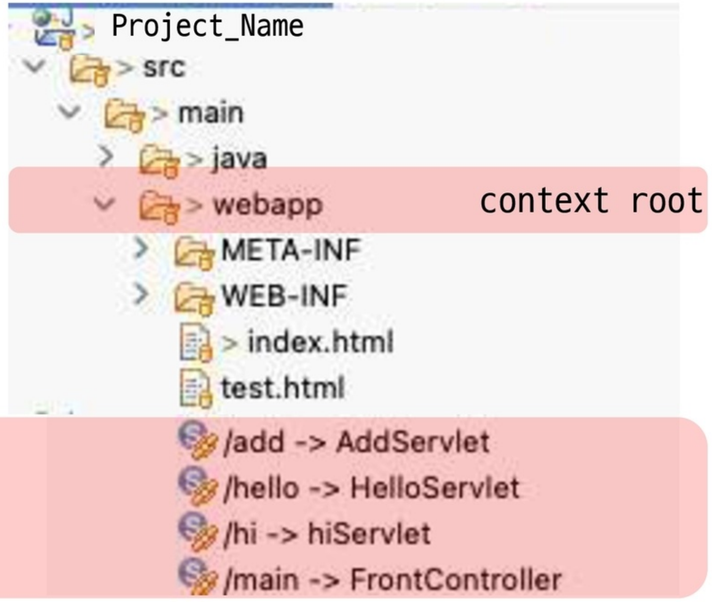
```
@WebServlet("/hello")
public class HelloServlet extends HttpServlet { }

@WebServlet(urlPatterns = { "/hello" })
public class HelloServlet extends HttpServlet {. . .}
```

Confidential

❖ 경로의 종류

- 상대 경로: /로 시작하지 않고 현재 파일의 위치 기반으로 계산하는 경로

- 절대 경로: / 로 시작하는 경로
  - ◆ container root: 컨테이너(WAS)의 경로
    - Servlet 배우기 전에 사용했던 ' / ' 는 container root
    - <a>, <link>, <form>, <script>의 경로 등
    - 특정 context를 호출하기 위해서는 /context_name/ 처럼 context_name을 개입시켜야 함
  - ◆ Context root: 개별 Context의 경로
    - servlet을 배우고 나서 배우는 경로는 '/'가 context root
    - servlet url mapping
    - web.xml의 경로
    - include-directive 의 경로 등

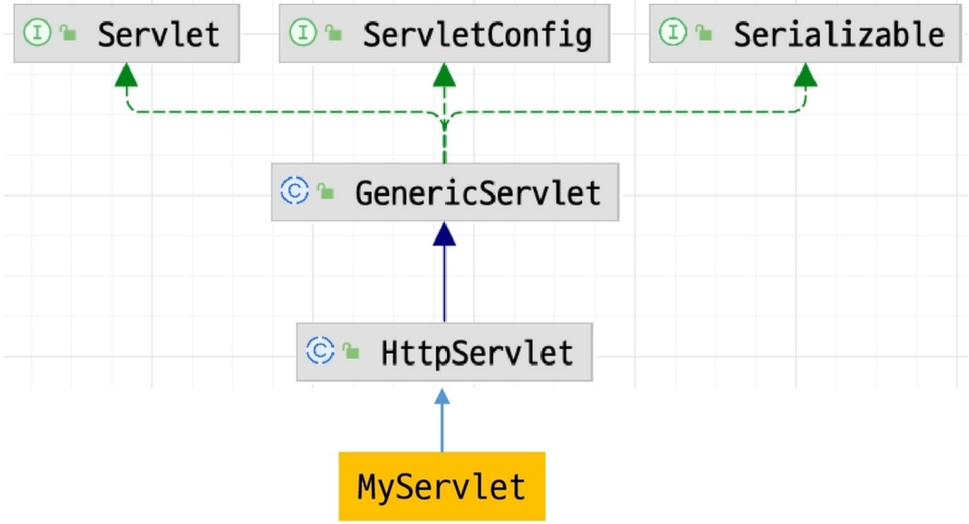


서블릿들도 논리적으로
이 경로에 매핑됨

Confidential

❖ Servlet 주요 API

- extends jakarta.servlet.http.HttpServlet
  - ◆ Jakarta EE의 패키지(과거 Java EE)
  - ◆ tomcat 등 웹 컨테이너들이 Jakarta EE 구현

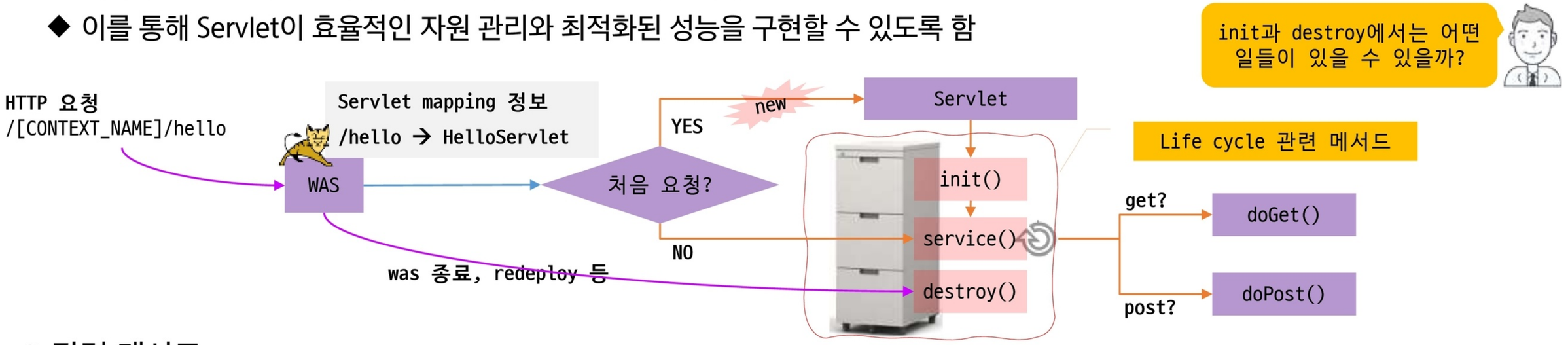


HttpServlet	
maybeSetLastModified(HttpServletResponse, long)	void
init(ServletConfig)	void
doPut(HttpServletRequest, HttpServletResponse)	void
doDelete(HttpServletRequest, HttpServletResponse)	void
isSensitiveHeader(String)	boolean
service(ServletRequest, ServletResponse)	void
doHead(HttpServletRequest, HttpServletResponse)	void
getAllDeclaredMethods(Class<?>)	Method[]
getLastModified(HttpServletRequest)	long
doOptions(HttpServletRequest, HttpServletResponse)	void
doTrace(HttpServletRequest, HttpServletResponse)	void
doGet(HttpServletRequest, HttpServletResponse)	void
getCachedAllowHeaderValue()	String
service(HttpServletRequest, HttpServletResponse)	void
sendMethodNotAllowed(HttpServletRequest, HttpServletResponse, St	
doPost(HttpServletRequest, HttpServletResponse)	void

Confidential

❖ Servlet Life Cycle 관리

- 개발자는 Servlet을 만들지만 객체를 만들거나 호출하지 않음 - Container가 life cycle에 따라 관리
  - ◆ 각각의 라이프 사이클 훅에서 할 일을 개발자가 적절히 작성 → Container가 호출
  - ◆ 이를 통해 Servlet이 효율적인 자원 관리와 최적화된 성능을 구현할 수 있도록 함



- 관련 메서드

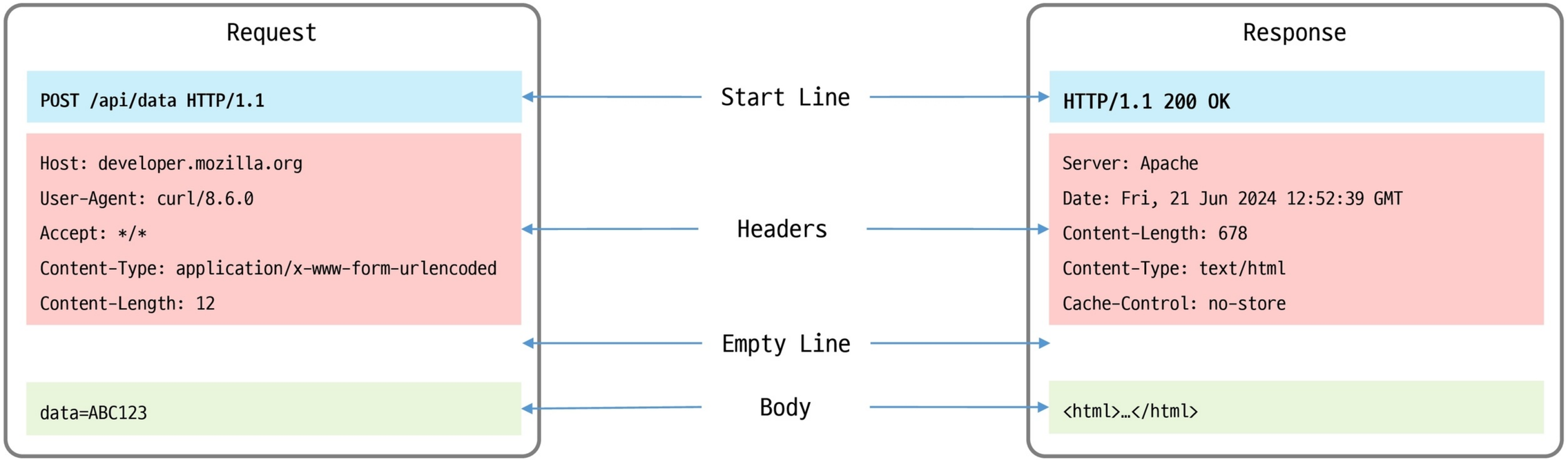
메서드명	설명	용도
init()	어떤 요청도 init이 종료되기 전에는 처리될 수 없음	Servlet에서 필요한 자원 초기화
service()	실제 사용자의 요청을 처리하는 메서드	요청 방식에 따라 doGet(), doPost() 호출
destroy()	어떤 요청이라도 처리하고 있으면 destroy는 동작하지 않음	init에서 초기화한 자원의 정리 작업

HttpServletRequest, HttpServletResponse

Confidential

❖ HttpServletRequest, HttpServletResponse

- 각각 Http의 Request, Response를 추상화하기 위한 인터페이스
- HTTP Request와 Response 포맷

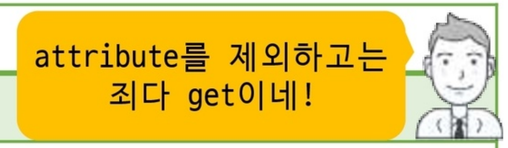


Confidential

❖ HttpServletRequest

- HTTP 요청을 추상화한 인터페이스로 헤더, 파라미터, 속성 및 요청 본문과 관련된 정보에 접근하는 메서드 제공

메서드	설명
String getMethod()	요청이 만들어진 HTTP 메서드의 이름을 반환
String getParameter(String name)	요청 파라미터의 값 반환
String[] getParameterValues(String name)	지정된 요청 파라미터의 모든 값을 포함하는 배열 반환
Enumeration<String> getParameterNames()	요청에 포함된 파라미터 이름의 열거형 반환
String getHeader(String name)	지정된 요청 헤더의 값을 반환
Enumeration<String> getHeaderNames()	요청에 포함된 모든 헤더 이름의 열거형 반환
void setAttribute(String name, Object o)	요청에 속성을 저장
Object getAttribute(String name)	요청에 저장된 속성의 값을 반환
HttpSession getSession()	현재 요청과 연관된 세션 반환



Request

```
POST /api/data HTTP/1.1
Host: developer.mozilla.org
User-Agent: curl/8.6.0
Accept: */*
Content-Type: application/x-www-form-urlencoded
Content-Length: 12

data=ABC123
```

Confidential

❖ HTTP method: form의 데이터를 서버로 전달하는 방식 결정

HTTP 메소드	RFC	요청에 Body가 있음	응답에 Body가 있음	안전	멱등(Idempotent)	캐시 가능
GET	RFC 9110	선택 사항	예	예	예	예
POST	RFC 9110	예	예	아니요	아니요	예
PUT	RFC 9110	예	예	아니요	예	아니요
DELETE	RFC 9110	선택 사항	예	아니요	예	아니요
CONNECT	RFC 9110	선택 사항	예	아니요	아니요	아니요
OPTIONS	RFC 9110	선택 사항	예	예	예	아니요
TRACE	RFC 9110	아니요	예	예	예	아니요
PATCH	RFC 5789	예	예	아니요	아니요	아니요
HEAD	RFC 9110	선택 사항	아니요	예	예	예

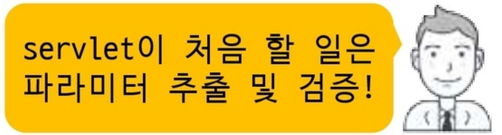
출처: https://ko.wikipedia.org/wiki/HTTP

- 멱등하다! : 특정 연산을 동일한 파라미터로 여러 번 연산하더라도 그 결과는 동일하다.

Confidential

❖ Request parameter

- 〈form〉 또는 queryString을 통해서 클라이언트가 전달한 값으로 **언제나 문자열**
  - ◆ 처음 클라이언트에서 설정된 이후 **조작은 불가**




```
getParameter(String arg0) : String - ServletRequest
getParameterMap() : Map<String,String[]> - ServletRequest
getParameterNames() : Enumeration<String> - ServletRequ
getParameterValues(String arg0) : String[] - ServletRequest
```

```
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
  System.out.println("doGet called: " + Thread.currentThread().getName());
  System.out.printf("##요청 경로: %s, 요청 방식: %s##\n", request.getRequestURI(), request.getMethod());
  System.out.println("요청 파라미터 분석");
  request.getParameterMap().forEach((k, v) -> {
    System.out.printf("name: %s, value: %s\n", k, Arrays.toString(v));
  });
  System.out.println("^^^
```

Confidential

❖ HttpServletResponse

- HTTP 응답을 추상화한 인터페이스로 헤더 조회/설정, 응답 상태, 응답 전송을 위한 작업 처리

메서드명	반환타입	설명
setStatus()	void	응답 상태 코드 설정
setHeader()	void	HTTP 응답 헤더 설정
setContentType()	void	응답 콘텐츠 타입과 함께 encoding도 설정
getWriter()	PrintWriter	문자 응답 스트림
addCookie()	void	HTTP 응답에 쿠키 추가
sendRedirect()	void	지정된 URL로 리다이렉트
sendError()	void	에러 응답 전송

Response

```
HTTP/1.1 200 OK

Server: Apache
Date: Fri, 21 Jun 2024 12:52:39 GMT
Content-Length: 678
Content-Type: text/html
Cache-Control: no-store


<html>…</html>
```

Confidential

❖ Http Status

- 웹 서버가 클라이언트의 요청을 처리한 결과를 카테고리 별로 지정된 숫자 코드로 반환

Category	Status Code	Name	Description
2xx Success	200	OK	요청의 정상 처리 상태
	201	Created	새로운 리소스 생성의 성공 상태
3xx Redirection	302	Moved Temporarily (Found)	요청한 리소스의 임시적 위치 이동 상태 (요청한 리소스가 다른 경로에서 발견 되었다. - found elsewhere)
4xx Client Error	400	Bad Request	잘못된 문법으로 인한 서버의 요청 처리 불가 상태
	401	Unauthorized	인증 실패 (당신이 누구인지 모르겠어!)
	403	Forbidden	권한 없음 (당신이 누구인지는 알지만 이건 못해!)
	404	Not Found	요청한 리소스를 찾을 수 없는 상태
	405	Method Not Allowed	허용되지 않은 HTTP 메서드 사용 상태
5xx Server Error	500	Internal Server Error	서버 내부 오류 발생 상태

Confidential

❖ Content-Type과 Character Encoding

● Content-Type

◆ 서버와 클라이언트가 데이터를 주고 받을 때 “이 데이터가 어떤 형식인지” 미리 알려주는 것

- 이를 통해 정보를 올바르게 해석 가능

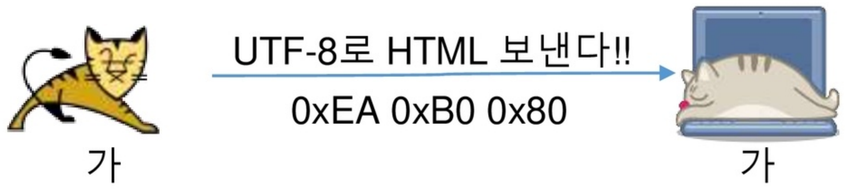
◆ 주요 content-type

Content-Type	용도	설명
text/html	HTML 문서	브라우저가 렌더링할 HTML 페이지
application/json	JSON 데이터	AJAX 통신에서 가장 많이 쓰이는 형식
image/jpeg	이미지 파일	바이너리 데이터를 그대로 전송
multipart/form-data	파일 업로드	텍스트 + 파일 혼합 전송

Confidential

❖ Content-Type과 Character Encoding

- Character Encoding
  - ◆ 데이터를 컴퓨터가 이해하고 처리할 수 있는 형태로 변환하는 방법
  - ◆ 응답의 기본 encoding은 ISO-8859-1(ASCII 확장) → 한글 표현에 부적합
  - ◆ 다시 원래의 문자로 변환(디코딩)하기 위해 인코딩 규칙 정보 필요



- 일반적으로 Content-Type과 Character Encoding을 함께 설정

```
response.setContentType("text/html;charset=UTF-8"); // response를 통해 출력하기 전에 설정!!
response.getWriter().append("<html><body>한글도 문제 없지</body></html>");
```

Servlet 작성 1

Confidential

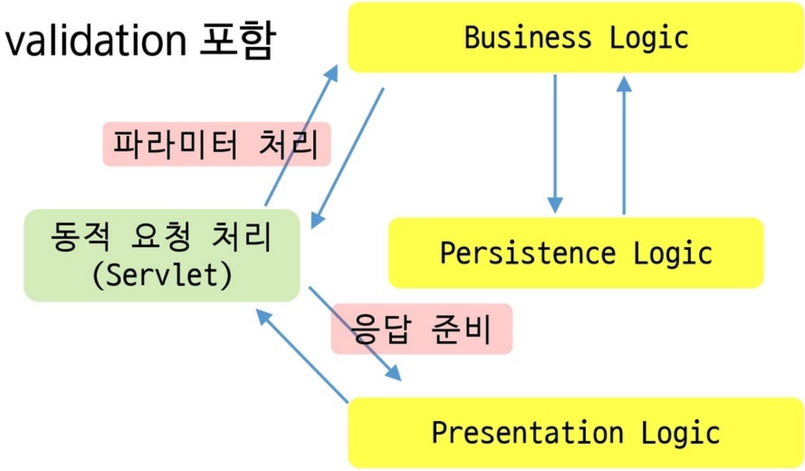
❖ AddServlet

- 두 개의 숫자형 파라미터를 받아서 더하기 결과를 반환하는 Servlet

```
<!-- index.html -->
<form action="/[CONTEXT_NAME]/add">
  <input type="number" name="num1" />+<input type="number" name="num2" /> = ?
  <button type="submit">더하기</button>
</form>
```

- url 매핑은?
- 재정의 할 메서드는 doGet/doPost?
- 전달되는 파라미터의 이름은?

- Servlet에서 할 일 정리
  - ◆ 요청 파라미터 조회: 파라미터에 대한 logging 및 validation 포함
  - ◆ 요청에 대한 business logic 처리
  - ◆ 필요 시 persistence logic 연동
  - ◆ 응답 준비: content type 및 encoding 설정
  - ◆ 응답을 위한 presentation logic 처리



Servlet 작성 2

Confidential

❖ LoginServlet

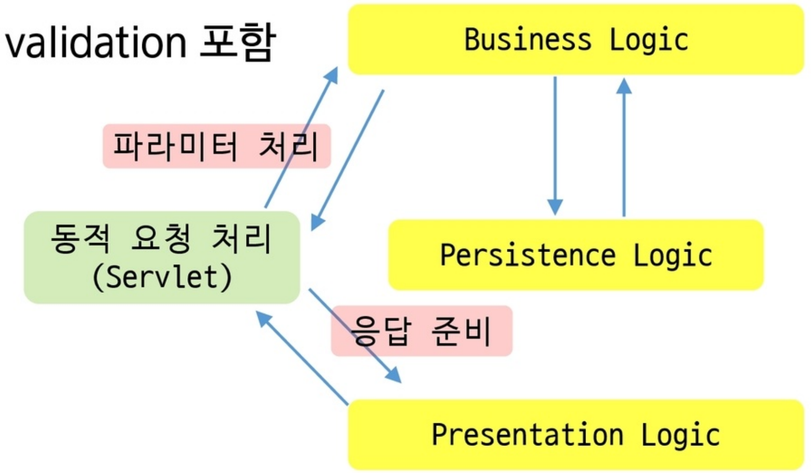
- id/pass를 파라미터로 받아서 로그인을 처리하는 Servlet

```
<form action="/[CONTEXT_NAME]/login" method="post">
  <input type="text" placeholder="아이디를 입력하세요" name="id" />
  <input type="password" placeholder="비밀번호를 입력하세요" name="pass" />
  <button>로그인</button>
</form>
```

- url 매핑은?
- 재정의 할 메서드는 doGet/doPost?
- 전달되는 파라미터의 이름은?

뭔가 공통적으로 진행되는 작업들이 있는걸?

- Servlet에서 할 일 정리
  - ◆ 요청 파라미터 조회: 파라미터에 대한 logging 및 validation 포함
  - ◆ 요청에 대한 business logic 처리
  - ◆ 필요 시 persistence logic 연동
  - ◆ 응답 준비: content type 및 encoding 설정
  - ◆ 응답을 위한 presentation logic 처리



내일 수업을 위해서..

Confidential

❖ 내일 수업을 위해서..

- 오늘 학습 내용 잘 정리하기

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

내일 방송에서 만나요!

삼성청년SW·AI아카데미